Modul Praktikum Pertemuan 3: Input dan Object Interaction pada WebXR

Prak. Augmented dan Virtual Reality (ITIA730591) · Program Studi Teknologi Informasi · FIKTI UMSU · 2026

1. Identitas Pertemuan

Komponen	Keterangan
Mata kuliah	Prak. Augmented dan Virtual Reality, kode ITIA730591, 1 sks praktik, semester VII
Pertemuan	3
Sub-CPMK	2.1: Mengimplementasikan input dan object interaction
Indikator	Select/grab/manipulate atau interaction ekuivalen bekerja
Waktu	Praktikum 1 x 170 menit
Teknik penilaian	Demo + rubrik (bobot 3%)
Luaran	Repository: interaction build
Materi	XR interaction basics
Tools	WebXR (Three.js), VS Code, Google Chrome/Microsoft Edge, Git

2. Tujuan Praktikum

Setelah praktikum ini, mahasiswa mampu:

1. Menjelaskan alur input pada WebXR: input source, controller, event select dan squeeze.
2. Membuat controller virtual lengkap dengan garis sinar (ray) pada scene WebXR.
3. Mengimplementasikan interaksi pada objek 3D: hover (sorot), select/grab (genggam), release (lepas), dan manipulate (ubah skala dan warna).
4. Menyediakan feedback visual dan haptic saat interaksi terjadi.

5. Menguji interaksi menggunakan emulator WebXR atau headset, lalu mengumpulkan hasilnya ke repository.

3. Alat dan Bahan

Perangkat lunak (wajib)

- Visual Studio Code, dengan ekstensi Live Server.
- Google Chrome atau Microsoft Edge versi terbaru.
- Git dan akun GitHub (repository dari Pertemuan 1).
- Ekstensi emulator WebXR untuk Chrome/Edge, misalnya Immersive Web Emulator. Dipakai jika headset tidak tersedia.
- Koneksi internet, karena Three.js dimuat dari CDN.

Perangkat keras (opsional)

- Headset yang mendukung WebXR, misalnya Meta Quest dengan Meta Quest Browser.
- Kabel USB-C dan ADB untuk pengujian lewat localhost, atau cukup memakai GitHub Pages.

Prasyarat

Scene dasar dari Pertemuan 1 dan 2 (project, struktur folder, objek, lighting, camera). Modul ini menyediakan scene minimal yang lengkap, jadi mahasiswa yang scene-nya belum selesai tetap bisa mengikuti. Mahasiswa yang scene-nya sudah jadi boleh memakai scene sendiri, asalkan objek yang bisa berinteraksi dimasukkan ke array `interactables`.

4. Konsep Singkat

- **Input source**: sumber input pada sesi XR, misalnya controller kiri/kanan atau tangan (hand tracking). Three.js membungkusnya sebagai objek controller.
- **Event `selectstart` / `selectend`**: aksi utama, biasanya tombol trigger. Dipakai untuk memilih dan menggenggam objek.
- **Event `squeezestart` / `squeezeend`**: aksi sekunder, biasanya tombol grip. Dipakai untuk aksi tambahan.
- **Raycasting**: sinar imajiner dari controller ke arah depan. Objek pertama yang terkena sinar menjadi target interaksi.
- **Grab**: objek dipindahkan menjadi anak (child) controller, sehingga ikut bergerak. Saat dilepas, objek dikembalikan ke scene dengan posisi dunia (world transform) yang dipertahankan.
- **Manipulate**: mengubah properti objek saat dipegang atau dipilih: skala, warna, rotasi.

- **Feedback**: respons yang membuat pengguna tahu interaksinya berhasil: sorot visual (emissive), getaran (haptic), dan log.

5. Alokasi Waktu (170 menit)

Kegiatan	Menit
Pembukaan, penjelasan konsep, dan tujuan	15
Persiapan project, server lokal, dan emulator	20
Scene dasar dan objek interaktif	30
Controller, ray, select, dan grab	35
Hover, squeeze, dan manipulasi skala	25
Pengujian dan perbaikan	15
Demo dan penilaian	15
Commit, push, dan dokumentasi	15

6. Langkah Praktikum

Langkah 1: Siapkan folder project dan Git (± 10 menit)

1. Buka folder repository praktikum Anda di VS Code.
2. Buat folder baru `pertemuan-3`. Di dalamnya buat dua file kosong: `index.html` dan `main.js`.
3. Buka terminal VS Code, lalu buat branch kerja:

```
git checkout -b pertemuan-3
```

Langkah 2: Isi `index.html` (± 5 menit)

Salin kode berikut ke `index.html`. Import map di bagian atas membuat Three.js dimuat dari CDN, jadi tidak perlu instalasi npm.

```
<!DOCTYPE html>
<html lang='id'>
<head>
  <meta charset='utf-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1'>
```

```
  <title>Praktikum AVR - Pertemuan 3</title>
  <style>
    body { margin: 0; overflow: hidden; font-family: sans-serif; }
    #log { position: fixed; top: 8px; left: 8px; right: 8px; color: #fff;
           font-size: 13px; pointer-events: none; }
  </style>
  <script type='importmap'>
  {
    "imports": {
      "three":
"https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js",
      "three/addons/":
"https://cdn.jsdelivr.net/npm/three@0.160.0/examples/jsm/"
    }
  }
  </script>
</head>
<body>
  <div id='log'>Log interaksi akan tampil di sini</div>
  <script type='module' src='main.js'></script>
</body>
</html>
```

Langkah 3: Jalankan server lokal dan siapkan emulator (± 10 menit)

WebXR hanya berjalan pada konteks aman (HTTPS atau `localhost`), jadi file tidak boleh dibuka langsung dengan klik ganda (`file://`).

1. Di VS Code, klik kanan `index.html`, pilih **Open with Live Server**. Browser membuka alamat seperti `http://127.0.0.1:5500/pertemuan-3/`.
2. Alternatif jika Live Server tidak ada, jalankan di folder `pertemuan-3`:

```
python -m http.server 8000
```

3. Pasang ekstensi emulator WebXR di Chrome/Edge (misalnya Immersive Web Emulator), lalu buka panel emulator tersebut di Developer Tools (F12).
4. Halaman masih kosong. Itu normal, isi `main.js` dikerjakan di langkah berikutnya.

Langkah 4: Tulis `main.js`, bagian A: import dan variabel (± 5 menit)

Bagian-bagian `main.js` (A sampai F) ditulis berurutan dari atas ke bawah dalam satu file. Pemanggilan `init();` diletakkan paling akhir di Langkah 9.

```
import * as THREE from 'three';
import { VRButton } from 'three/addons/webxr/VRButton.js';
import { XRControllerModelFactory } from
'three/addons/webxr/XRControllerModelFactory.js';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

let camera, scene, renderer, orbit;
const controllers = [];
const interactables = [];   // daftar objek yang boleh berinteraksi
const raycaster = new THREE.Raycaster();
const tempMatrix = new THREE.Matrix4();
const pointer = new THREE.Vector2();
const logBox = document.getElementById('log');
const lines = [];

// Log ke layar dan console, berguna sebagai bukti pengujian
function log(msg) {
  console.log(msg);
  lines.unshift(msg);
  lines.length = Math.min(lines.length, 5);
  logBox.textContent = lines.join('  |  ');
}
```

Langkah 5: Bagian B: scene dan objek interaktif (± 25 menit)

Tambahkan di bawah kode sebelumnya. Fungsi `init()` menyiapkan scene, lighting, lantai, tiga objek, renderer, dan tombol masuk VR.

```
function init() {
  scene = new THREE.Scene();
  scene.background = new THREE.Color(0x1b1f2a);

  camera = new THREE.PerspectiveCamera(60, window.innerWidth /
window.innerHeight, 0.1, 50);
  camera.position.set(0, 1.6, 2);

  scene.add(new THREE.HemisphereLight(0xffffff, 0x444455, 1.5));
  const sun = new THREE.DirectionalLight(0xffffff, 1.5);
  sun.position.set(2, 4, 2);
  scene.add(sun);

  const floor = new THREE.Mesh(
    new THREE.PlaneGeometry(10, 10),
    new THREE.MeshStandardMaterial({ color: 0x2e3447 })
  );
```

```
  floor.rotation.x = -Math.PI / 2;
  scene.add(floor);
  scene.add(new THREE.GridHelper(10, 20, 0x555a6e, 0x3a3f52));

  // Tiga objek yang bisa disentuh/digenggam
  addObject(new THREE.BoxGeometry(0.3, 0.3, 0.3), 0xe74c3c, -0.6, 1.2, -1);
  addObject(new THREE.SphereGeometry(0.18, 32, 16), 0x3498db, 0, 1.2, -1);
  addObject(new THREE.ConeGeometry(0.16, 0.35, 32), 0x2ecc71, 0.6, 1.2, -1);

  renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setPixelRatio(window.devicePixelRatio);
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.xr.enabled = true;               // aktifkan WebXR
  document.body.appendChild(renderer.domElement);
  document.body.appendChild(VRButton.createButton(renderer));

  // Kontrol mouse untuk mode desktop (tidak aktif saat sesi XR)
  orbit = new OrbitControls(camera, renderer.domElement);
  orbit.target.set(0, 1.2, -1);
  orbit.update();

  setupControllers();
  window.addEventListener('resize', onResize);
  renderer.domElement.addEventListener('pointerdown', onPointerDown);
  renderer.setAnimationLoop(render);
}

function addObject(geometry, color, x, y, z) {
  const mesh = new THREE.Mesh(geometry, new THREE.MeshStandardMaterial({
color }));
  mesh.position.set(x, y, z);
  scene.add(mesh);
  interactables.push(mesh);
}

function onResize() {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
}
```

Cek sementara: simpan file, lalu tambahkan `init();` di baris paling bawah `main.js` untuk uji awal (Langkah 9 membahas posisi akhirnya). Muat ulang browser. Anda harus melihat lantai, grid, dan tiga objek, dan bisa memutar kamera dengan mouse. Tombol **ENTER VR** muncul di bagian bawah layar. Jika ada error, buka Console (F12) dan baca pesannya sebelum lanjut.

Langkah 6: Bagian C: controller dan garis sinar (± 15 menit)

Tambahkan di bawah kode sebelumnya. Setiap controller mendapat garis sinar dan model 3D controller asli.

```
function setupControllers() {
  const modelFactory = new XRControllerModelFactory();

  for (let i = 0; i < 2; i++) {
    const controller = renderer.xr.getController(i);

    controller.addEventListener('connected', (e) => {
      controller.userData.gamepad = e.data.gamepad;   // untuk haptic dan
thumbstick
      controller.add(buildRay());
      log('Controller ' + i + ' terhubung (' + e.data.handedness + ')');
    });
    controller.addEventListener('disconnected', () => controller.clear());
    controller.addEventListener('selectstart', onSelectStart);
    controller.addEventListener('selectend', onSelectEnd);
    controller.addEventListener('squeezestart', onSqueezeStart);

    scene.add(controller);
    controllers.push(controller);

    const grip = renderer.xr.getControllerGrip(i);
    grip.add(modelFactory.createControllerModel(grip));
    scene.add(grip);
  }
}

function buildRay() {
  const geometry = new THREE.BufferGeometry().setFromPoints([
    new THREE.Vector3(0, 0, 0),
    new THREE.Vector3(0, 0, -1)
  ]);
  const line = new THREE.Line(geometry, new THREE.LineBasicMaterial({ color:
0xffffff }));
  line.name = 'ray';
  line.scale.z = 5;   // panjang sinar 5 meter
  return line;
}
```

Langkah 7: Bagian D: raycast, select/grab, release, squeeze (± 20 menit)

Ini inti pertemuan ini. Tambahkan di bawah kode sebelumnya.

```
// Menghitung objek yang terkena sinar dari sebuah controller
function getIntersections(controller) {
  tempMatrix.identity().extractRotation(controller.matrixWorld);
  raycaster.ray.origin.setFromMatrixPosition(controller.matrixWorld);
  raycaster.ray.direction.set(0, 0, -1).applyMatrix4(tempMatrix);
  return raycaster.intersectObjects(interactables, false);
}

// Getaran singkat pada controller (jika perangkat mendukung)
function pulse(controller, strength = 0.5, ms = 80) {
  const gp = controller.userData.gamepad;
  const actuator = gp && gp.hapticActuators && gp.hapticActuators[0];
  if (actuator && actuator.pulse) actuator.pulse(strength, ms);
}

// Trigger ditekan: genggam objek yang terkena sinar
function onSelectStart(event) {
  const controller = event.target;
  const hits = getIntersections(controller);
  if (hits.length > 0) {
    const obj = hits[0].object;
    controller.attach(obj);            // objek menjadi anak controller
    controller.userData.selected = obj;
    pulse(controller);
    log('GRAB: ' + obj.geometry.type);
  }
}

// Trigger dilepas: kembalikan objek ke scene di posisi terakhirnya
function onSelectEnd(event) {
  const controller = event.target;
  const obj = controller.userData.selected;
  if (obj) {
    scene.attach(obj);                 // posisi dunia tetap terjaga
    controller.userData.selected = undefined;
    log('RELEASE: ' + obj.geometry.type);
  }
}

// Grip ditekan: ganti warna objek yang disorot
function onSqueezeStart(event) {
  const controller = event.target;
  const hits = getIntersections(controller);
  if (hits.length > 0) {
    hits[0].object.material.color.setHex(Math.floor(Math.random() *
0xffffff));
```

```
    pulse(controller, 0.3, 50);
    log('SQUEEZE: warna diubah');
  }
}
```

Perbedaan `attach()` dan `add()`: `attach()` mempertahankan posisi, rotasi, dan skala objek di dunia, jadi objek tidak melompat saat berpindah induk. Dengan `add()`, objek akan melompat.

Langkah 8: Bagian E dan F: hover, manipulasi skala, loop render, dan fallback desktop (± 20 menit)

Tambahkan di bawah kode sebelumnya.

```
function render() {
  // Reset sorotan semua objek
  interactables.forEach((o) => o.material.emissive.setHex(0x000000));

  controllers.forEach((c) => {
    const held = c.userData.selected;

    if (held) {
      // Objek sedang digenggam: sorot dan izinkan ubah skala dengan
thumbstick
      held.material.emissive.setHex(0x222222);
      const gp = c.userData.gamepad;
      if (gp && gp.axes.length >= 4) {
        const y = gp.axes[3];                // sumbu Y thumbstick
        if (Math.abs(y) > 0.15) {            // abaikan drift kecil
          held.scale.multiplyScalar(1 - y * 0.02).clampScalar(0.3, 3);
        }
      }
    } else if (c.children.length > 0) {
      // Controller sudah terhubung dan tidak memegang apa pun: cek hover
      const hits = getIntersections(c);
      if (hits.length > 0) hits[0].object.material.emissive.setHex(0x444444);
    }
  });

  if (!renderer.xr.isPresenting) orbit.update();
  renderer.render(scene, camera);
}

// Interaksi ekuivalen untuk mode desktop: klik mouse mengganti warna objek
function onPointerDown(e) {
  if (renderer.xr.isPresenting) return;
```

```
  pointer.set(
    (e.clientX / window.innerWidth) * 2 - 1,
    -(e.clientY / window.innerHeight) * 2 + 1
  );
  raycaster.setFromCamera(pointer, camera);
  const hits = raycaster.intersectObjects(interactables, false);
  if (hits.length > 0) {
    hits[0].object.material.color.setHex(Math.floor(Math.random() *
0xffffff));
    log('CLICK (desktop): warna diubah');
  }
}
```

Langkah 9: Panggil `init()` dan jalankan (± 5 menit)

1. Pastikan baris `init();` ada **sekali saja** di baris paling bawah `main.js`. Hapus pemanggilan `init();` sementara dari Langkah 5 jika masih ada di tempat lain.
2. Simpan semua file, lalu muat ulang halaman di browser.
3. Di mode desktop, klik salah satu objek. Warnanya harus berubah dan log tampil di bagian atas layar.

Langkah 10: Uji di emulator WebXR (± 10 menit)

1. Buka panel emulator WebXR di Developer Tools (F12).
2. Klik tombol **ENTER VR** di halaman. Emulator memulai sesi XR, dan controller virtual muncul di scene.
3. Gerakkan controller virtual sampai sinarnya mengenai sebuah objek. Objek harus menyala redup (hover).
4. Tekan tombol trigger pada controller emulator. Objek harus ikut bergerak bersama controller (GRAB). Lepas trigger, objek harus berhenti di posisi terakhir (RELEASE).
5. Tekan tombol grip (squeeze) saat sinar mengenai objek. Warnanya harus berubah.
6. Saat menggenggam objek, geser thumbstick maju dan mundur. Objek harus membesar dan mengecil (jika emulator mensimulasikan thumbstick).
7. Ambil tangkapan layar atau rekam layar 30 sampai 60 detik yang memperlihatkan hover, grab, release, dan squeeze.

Langkah 11: Uji di headset (opsional, ± 10 menit)

Pilih salah satu cara.

Cara 1, GitHub Pages (paling mudah):

1. Push project ke GitHub (Langkah 13).

2. Aktifkan GitHub Pages pada repository (Settings, lalu Pages), pilih branch yang berisi folder `pertemuan-3`.
3. Buka alamat HTTPS-nya di Meta Quest Browser, lalu tekan **ENTER VR**.

Cara 2, USB dan ADB (lewat localhost):

1. Aktifkan Developer Mode pada headset, hubungkan dengan kabel USB, dan izinkan debugging.
2. Jalankan di komputer (sesuaikan nomor port dengan server lokal Anda):

```
adb reverse tcp:5500 tcp:5500
```

3. Di Meta Quest Browser, buka `http://localhost:5500/pertemuan-3/`, lalu tekan **ENTER VR**.

Langkah 12: Isi tabel uji fungsional (± 5 menit)

Catat hasil pengujian Anda pada tabel berikut dan simpan di `pertemuan-3/README.md`.

No	Skenario uji	Hasil yang diharapkan	Hasil aktual	Status
1	Buka halaman di browser	Scene dan 3 objek tampil, tombol ENTER VR muncul		
2	Masuk sesi XR	Controller dan garis sinar tampil		
3	Arahkan sinar ke objek	Objek menyala redup (hover)		
4	Tekan dan tahan trigger pada objek	Objek ikut controller (grab)		
5	Lepas trigger	Objek tetap di posisi terakhir (release)		
6	Tekan grip pada objek	Warna objek berubah (squeeze)		
7	Geser thumbstick saat objek digenggam	Skala objek berubah, dibatasi 0,3 sampai 3 kali		
8	Klik objek dengan mouse (desktop)	Warna objek berubah		

Langkah 13: Commit dan push (± 10 menit)

```
git add pertemuan-3
git commit -m "P3: interaction build (hover, grab, release, squeeze, scale)"
git push origin pertemuan-3
```

Buka repository di GitHub, pastikan folder `pertemuan-3` ada, lalu buat Pull Request ke branch `main` atau merge sesuai petunjuk dosen. Tambahkan tag penanda:

```
git tag pertemuan-3
git push origin pertemuan-3
```

7. Pemecahan Masalah

Gejala	Kemungkinan penyebab	Solusi
Layar kosong, Console menampilkan error modul	Halaman dibuka dengan `file://` atau CDN tidak terjangkau	Jalankan lewat Live Server atau `python -m http.server`, dan periksa koneksi internet
Tombol VR bertuliskan "VR NOT SUPPORTED"	Tidak ada emulator atau headset	Pasang ekstensi emulator WebXR, atau pakai headset
Tombol VR bertuliskan "WEBXR NEEDS HTTPS"	Halaman diakses lewat IP jaringan dengan HTTP	Pakai `localhost`, GitHub Pages, atau `adb reverse`
Controller tidak muncul	Sesi XR belum dimulai	Tekan ENTER VR dulu, lalu cek log "Controller ... terhubung"
Objek melompat saat digenggam	Memakai `add()` bukan `attach()`	Gunakan `controller.attach(obj)` dan `scene.attach(obj)`
Objek tidak bisa dipilih	Objek belum masuk array `interactables`	Buat objek lewat `addObject()` atau `interactables.push(mesh)`
Skala tidak berubah	Thumbstick tidak terbaca emulator, atau indeks sumbu berbeda	Cek `gp.axes` di console, lalu sesuaikan indeks (umumnya 2 dan 3 untuk thumbstick)
Tidak ada getaran	Perangkat atau emulator tidak mendukung haptic	Normal. Fungsi `pulse()` sudah aman bila haptic tidak tersedia

8. Tugas Pengayaan (nilai tambah)

Pilih minimal satu:

1. **Drop zone**: tambahkan area di lantai (misalnya kotak transparan). Saat objek dilepas di dalam area tersebut, ubah warna area menjadi hijau dan catat di log.
2. **Rotasi objek**: gunakan sumbu X thumbstick (`gp.axes[2]`) untuk memutar objek yang digenggam.
3. **Reset scene**: tombol pada controller (misalnya `gp.buttons[4]`) mengembalikan semua objek ke posisi awal.
4. **Tambah objek**: buat tiga objek baru dengan geometri berbeda, semuanya interaktif.

9. Rubrik Penilaian (Demo + Rubrik)

Aspek	Kriteria	Skor maks
Setup dan struktur repository	Project berjalan, folder `pertemuan-3` rapi, commit bermakna	15
Controller dan hover	Controller dan sinar tampil, objek menyala saat disorot	20
Select/grab dan release	Objek bisa digenggam dan dilepas tanpa melompat	25
Manipulate	Minimal satu manipulasi berfungsi (skala dan/atau warna)	20
Demo dan penjelasan	Mahasiswa mampu menjelaskan alur event dan raycasting	10
Dokumentasi dan bukti uji	Tabel uji fungsional terisi, ada tangkapan layar atau video	10
Total		**100**

Nilai akhir pertemuan dikonversi ke bobot 3% sesuai RPS.

10. Ketentuan Pengumpulan

- Tautan repository GitHub berisi folder `pertemuan-3` (kode, README.md dengan tabel uji, tangkapan layar atau video demo).
- Commit dan tag `pertemuan-3` tersedia di repository.

- Demo singkat langsung kepada dosen pada akhir sesi praktikum.

11. Referensi

1. Jerald, J. (2015). The VR Book: Human-Centered Design for Virtual Reality. ACM Books/Morgan & Claypool.
2. LaValle, S. M. (2017). Virtual Reality. Cambridge University Press.
3. Dokumentasi resmi WebXR Device API (MDN Web Docs) dan Three.js (threejs.org/docs), khususnya WebXR dan Raycaster.